

ESEG – Grupo ETAPA
Engenharia da Computação

Ramon Vieira
Nícolas Pereira
Vinicius Venancio

FACEGATEWAY: SISTEMA DE CONTROLE DE ENTRADA E SAÍDA
BASEADO EM RECONHECIMENTO FACIAL

Projeto de Sistema Embarcado com Processamento e Análise Logística

SÃO PAULO
2026

Ramon Vieira
Nícolas Pereira
Vinicius Venancio

**FACEGATEWAY: SISTEMA DE CONTROLE DE ENTRADA E SAÍDA
BASEADO EM RECONHECIMENTO FACIAL**

Trabalho apresentado ao Curso de Engenharia da
Computação na ESEG como requisito parcial para
obtenção da nota na disciplina Computação Embarcada.

Professor: Filippo Filho

Sumário

1	Introdução	1
1.1	Contextualização	1
1.2	Problema de Pesquisa	1
1.3	Objetivos	2
1.3.1	Objetivo Geral	2
1.3.2	Objetivos Específicos	2
1.4	Justificativa	2
2	Fundamentação Teórica	4
2.1	Reconhecimento Facial	4
2.2	Edge Computing e Sistemas Embarcados	4
2.3	Protocolos de Comunicação IoT	5
2.4	Aspectos Legais e Éticos	5
3	Arquitetura do Sistema	6
3.1	Visão Geral	6
3.2	Módulo Embarcado – Raspberry Pi 4	7
3.3	Backend – Servidor Local	8
3.4	Painel Web e Mobile	8
4	Organização do Projeto	9
4.1	Divisão de Tarefas	9
4.2	Cronograma – Diagrama de Gantt	9
4.3	Estimativa de Custos	13
5	Metodologia	14
5.1	Coleta de Dados e Cadastro de Funcionários	14
5.2	Definição do Limiar de Autenticação	14
5.3	Testes e Avaliação	14
6	Desenvolvimento	16
6.1	Implementação do Sistema	16

6.2	Integração dos Módulos	16
6.3	Desafios Técnicos	16
7	Resultados	17
7.1	Desempenho do Modelo	17
7.2	Tempo de Processamento	17
7.3	Análise Logística Gerada	17
8	Discussão	18
9	Conclusão	19
	Referências	20
A	Especificações Técnicas	21
B	Trechos Relevantes de Código	22

Lista de Figuras

3.1	Diagrama geral da arquitetura do FaceGateway	6
4.1	Diagrama de Gantt – FaceGateway (março a maio de 2026)	12

Lista de Tabelas

4.1	Divisão de responsabilidades por integrante	9
4.2	Marcos do projeto	10
4.3	Estimativa de custo do hardware para o MVP	13
A.1	Stack tecnológico completo	21

Capítulo 1

Introdução

1.1 Contextualização

O controle de entrada e saída de funcionários é uma atividade essencial para organizações que operam com turnos fixos ou escalas de trabalho. Métodos tradicionais, como cartões magnéticos, senhas ou registros manuais, apresentam limitações relacionadas à segurança, possibilidade de fraudes e baixa integração com sistemas analíticos.

Com o avanço das técnicas de Visão Computacional e Aprendizado de Máquina, especialmente no campo do reconhecimento facial, tornou-se viável implementar soluções biométricas com processamento local e custo relativamente reduzido. Dispositivos embarcados de baixo consumo energético, como o Raspberry Pi 4, permitem a execução de modelos de aprendizado de máquina diretamente no ponto de captura, reduzindo latência e dependência de infraestrutura centralizada.

Além do controle de acesso, a coleta estruturada desses dados possibilita análises logísticas relevantes, como cálculo de tempo médio de permanência, identificação de padrões de atraso e avaliação de horários de entrada e saída. Essas informações podem apoiar decisões estratégicas relacionadas à gestão de recursos humanos.

1.2 Problema de Pesquisa

Como implementar um sistema de controle de acesso baseado em reconhecimento facial utilizando hardware de baixo custo, garantindo desempenho adequado, confiabilidade na identificação e geração de métricas logísticas úteis para análise organizacional, mantendo os dados biométricos restritos ao dispositivo embarcado?

1.3 Objetivos

1.3.1 Objetivo Geral

Desenvolver um sistema de controle de entrada e saída de funcionários baseado em reconhecimento facial, com processamento embarcado em C++ na Raspberry Pi 4 e backend analítico em Rust, expondo métricas logísticas via painel web e mobile.

1.3.2 Objetivos Específicos

- Implementar pipeline de visão computacional em C++ capaz de detectar e reconhecer rostos em tempo real, com latência inferior a 500 ms.
- Garantir operação offline completa – catraca e buzzer funcionam independentemente de conectividade de rede.
- Transmitir eventos de acesso ao backend via protocolo MQTT com garantia de entrega (QoS 1).
- Desenvolver API REST em Rust (Axum) com autenticação JWT e endpoints de analytics.
- Persistir embeddings e eventos no PostgreSQL com extensão pgvector para busca por similaridade.
- Desenvolver painel multiplataforma em Ionic + Angular, instalável como Progressive Web App.
- Estimar e documentar os custos de hardware necessários para implantação do MVP.

1.4 Justificativa

A adoção de sistemas biométricos no controle de acesso reduz vulnerabilidades associadas a métodos tradicionais, como empréstimo de cartões ou compartilhamento de credenciais. O reconhecimento facial elimina a necessidade de contato físico e integra-se de forma transparente ao fluxo de entrada e saída.

Do ponto de vista técnico, o projeto demonstra a viabilidade de execução de modelos de aprendizado de máquina em dispositivos embarcados de baixo custo, explorando otimização de desempenho com o formato ONNX e integração entre hardware e software em C++.

Uma decisão central de projeto é manter o reconhecimento facial completamente *on-device*: nenhuma imagem bruta ou dado biométrico trafega pela rede. Apenas metadados de eventos (identificador do funcionário, horário, resultado) são enviados ao servidor,

contribuindo diretamente para conformidade com princípios de minimização de dados da LGPD.

Capítulo 2

Fundamentação Teórica

2.1 Reconhecimento Facial

O reconhecimento facial é uma área da Visão Computacional dedicada à identificação ou verificação da identidade de um indivíduo a partir de características faciais extraídas de imagens ou vídeos. Sistemas modernos operam em três etapas principais: detecção facial, extração de características e comparação.

A detecção consiste na identificação da presença de um rosto em uma imagem. Neste projeto, utiliza-se o modelo **BlazeFace** (Google MediaPipe), exportado no formato ONNX, que além das *bounding boxes* fornece *keypoints* faciais – posições dos olhos, nariz e boca – essenciais para o alinhamento do rosto antes da geração do embedding.

A extração de características utiliza o modelo **MobileFaceNet**, treinado com a função de perda ArcFace no dataset MS-Celeb. O modelo gera embeddings de 128 dimensões que capturam as características discriminativas do rosto. A identificação ocorre pela comparação da distância de cosseno entre o embedding extraído e os embeddings previamente armazenados: valores abaixo de um limiar ($\approx 0,45$) indicam correspondência positiva.

2.2 Edge Computing e Sistemas Embarcados

O conceito de *Edge Computing* destaca a importância do processamento local, reduzindo dependência de servidores remotos e minimizando latência. No caso de controle de acesso, o processamento embarcado permite resposta imediata à tentativa de autenticação, além de evitar a exposição de dados biométricos em redes externas.

A execução de modelos de aprendizado de máquina em hardware limitado é viabilizada pelo formato **ONNX** (*Open Neural Network Exchange*), que permite exportar modelos treinados em qualquer framework e executá-los via **ONNXRuntime** com API C++, otimizando o uso de recursos disponíveis na Raspberry Pi 4.

2.3 Protocolos de Comunicação IoT

O protocolo **MQTT** (*Message Queuing Telemetry Transport*) é o padrão de mercado para comunicação em dispositivos IoT. Seu modelo *publish/subscribe* desacopla produtor e consumidor de mensagens, e os níveis de qualidade de serviço (QoS) garantem entrega mesmo em redes instáveis. No nível QoS 1, o broker retém mensagens enquanto o subscriber estiver offline, garantindo que nenhum evento de acesso seja perdido.

2.4 Aspectos Legais e Éticos

Informações biométricas são classificadas como dados sensíveis pela Lei Geral de Proteção de Dados (LGPD – Lei 13.709/2018), pois estão diretamente associadas à identidade do indivíduo. Entre os princípios aplicáveis ao projeto destacam-se:

- Finalidade específica e legítima para coleta dos dados;
- Minimização: armazenamento de embeddings, não de imagens brutas;
- Transparência quanto ao uso das informações coletadas;
- Garantia de segurança e confidencialidade dos dados armazenados.

A decisão de manter o reconhecimento facial exclusivamente *on-device* é diretamente motivada por esses princípios, eliminando o risco de vazamento de dados biométricos em trânsito.

Capítulo 3

Arquitetura do Sistema

3.1 Visão Geral

O sistema é organizado em três camadas físicas e lógicas independentes: o módulo embarcado (Raspberry Pi 4), o servidor local (notebook do desenvolvedor, substituível por nuvem em produção) e o painel web/mobile. A Figura 3.1 apresenta o diagrama geral da arquitetura.

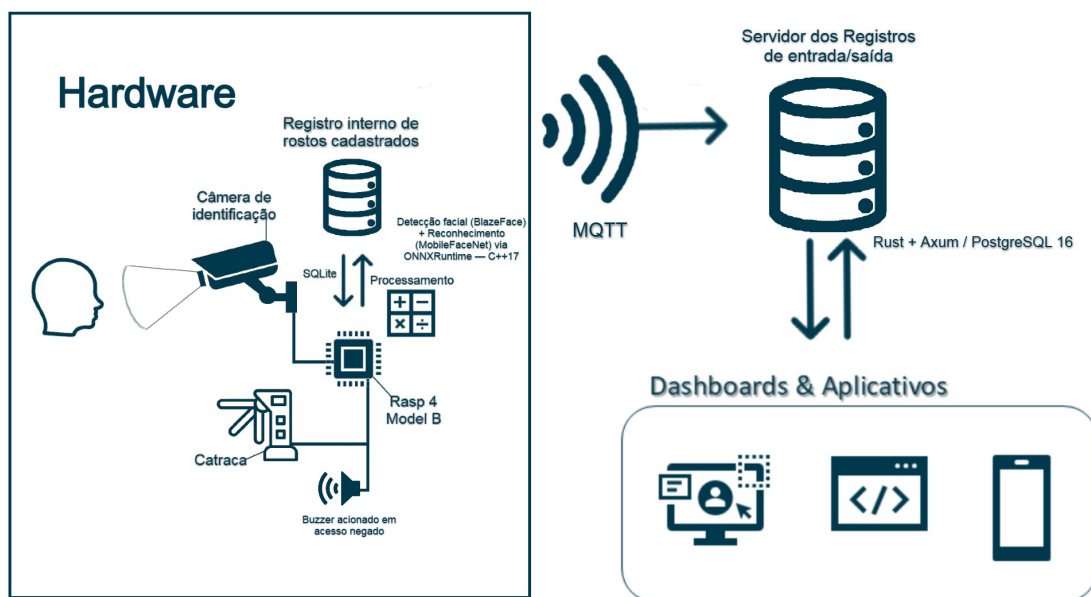


Figura 3.1: Diagrama geral da arquitetura do FaceGateway

O fluxo operacional completo ocorre nas seguintes etapas:

1. Câmera CSI captura frames continuamente.
2. BlazeFace detecta presença de rosto e extrai *keypoints* para alinhamento.
3. MobileFaceNet gera embedding de 128 dimensões do rosto alinhado.

4. Embedding é comparado por distância de cosseno com o cache local SQLite.
5. Decisão: acesso concedido aciona relé da catraca via GPIO; acesso negado aciona buzzer.
6. Evento é publicado no broker MQTT (tópico `facegate/events/access`).
7. Subscriber Rust consome o evento e persiste no PostgreSQL.
8. Painel Ionic + Angular consulta a API REST e exibe métricas ao gestor.

O módulo embarcado opera de forma completamente autônoma: em caso de falha de rede, eventos são enfileirados em SQLite local e enviados ao broker quando a conectividade é restabelecida.

3.2 Módulo Embarcado – Raspberry Pi 4

O módulo embarcado é responsável por toda a inteligência de visão computacional e atuação física. O hardware é composto por:

- Raspberry Pi 4 Model B (4 GB de RAM);
- Câmera CSI de 5 megapixels;
- Relé para acionamento da catraca (via GPIO);
- Buzzer para sinalização de acesso negado (via GPIO).

O software embarcado é desenvolvido em **C++17** e utiliza as seguintes bibliotecas:

- **libcamera**: captura de frames via interface CSI;
- **OpenCV 4.8**: pré-processamento de imagem;
- **ONNXRuntime 1.17**: inferência dos modelos BlazeFace e MobileFaceNet;
- **Eigen3**: cálculo de distância de cosseno entre embeddings;
- **SQLite3**: cache local de embeddings e fila de eventos offline;
- **libmosquitto**: publicação de eventos no broker MQTT.

3.3 Backend – Servidor Local

O backend é desenvolvido em **Rust** e composto pelos seguintes componentes:

- **Axum 0.7**: framework HTTP para a API REST;
- **sqlx 0.7**: queries verificadas em *compile-time* contra o PostgreSQL;
- **rumqtte 0.24**: subscriber MQTT assíncrono integrado ao runtime Tokio;
- **jsonwebtoken**: geração e validação de tokens JWT para autenticação do painel.

O banco de dados utilizado é o **PostgreSQL 16** com a extensão **pgvector 0.6**, que permite armazenar embeddings vetoriais e realizar busca por similaridade com índice HNSW diretamente no banco, eliminando a necessidade de biblioteca externa como FAISS.

A comunicação entre Raspberry e backend utiliza o protocolo **MQTT com QoS 1**, com broker **Eclipse Mosquitto 2.x** rodando no servidor. A estrutura de tópicos é:

- `facegate/events/access` – evento de rosto reconhecido;
- `facegate/events/unknown` – evento de rosto não reconhecido;
- `facegate/health` – *heartbeat* periódico do dispositivo.

3.4 Painel Web e Mobile

O painel é desenvolvido em **Ionic 7 + Angular 17**, gerando uma aplicação multiplataforma instalável como **Progressive Web App** (PWA) – funciona no browser do desktop e pode ser instalado no celular como app nativo sem necessidade de loja de aplicativos.

As telas planejadas para o MVP são:

- **Login**: autenticação JWT com *guard* de rotas protegidas;
- **Dashboard**: cards de resumo e gráfico de fluxo de acessos por hora (ngx-echarts);
- **Funcionários**: listagem, cadastro e upload de foto para geração de embedding;
- **Histórico de eventos**: tabela paginada com filtros por funcionário e data;
- **Analytics**: atraso médio por funcionário e heatmap de presença por dia/hora.

Capítulo 4

Organização do Projeto

4.1 Divisão de Tarefas

O grupo é composto por quatro integrantes com as seguintes responsabilidades principais:

Tabela 4.1: Divisão de responsabilidades por integrante

Integrante	Foco principal	Responsabilidades
Ramon	Edge C++ + Rust crítico	Setup toolchain Raspberry, pipeline de visão (BlazeFace, MobileFaceNet, Eigen3), GPIO, autenticação JWT, sync de embeddings, ajuste de threshold
Nícolas	Backend + banco	Setup PostgreSQL/pgvector, Mosquitto, subscriber MQTT, schema do banco, CRUD de funcionários, endpoints de analytics
Kaique C	Suporte edge + integração	Buzzer GPIO, fila offline SQLite, apoio ao CRUD, tela de histórico de eventos, testes de integração, documentação
Vinicius D	Frontend + design	Setup Ionic/Angular/PWA, tela de login, tela de funcionários, dashboard com gráficos, tela de analytics

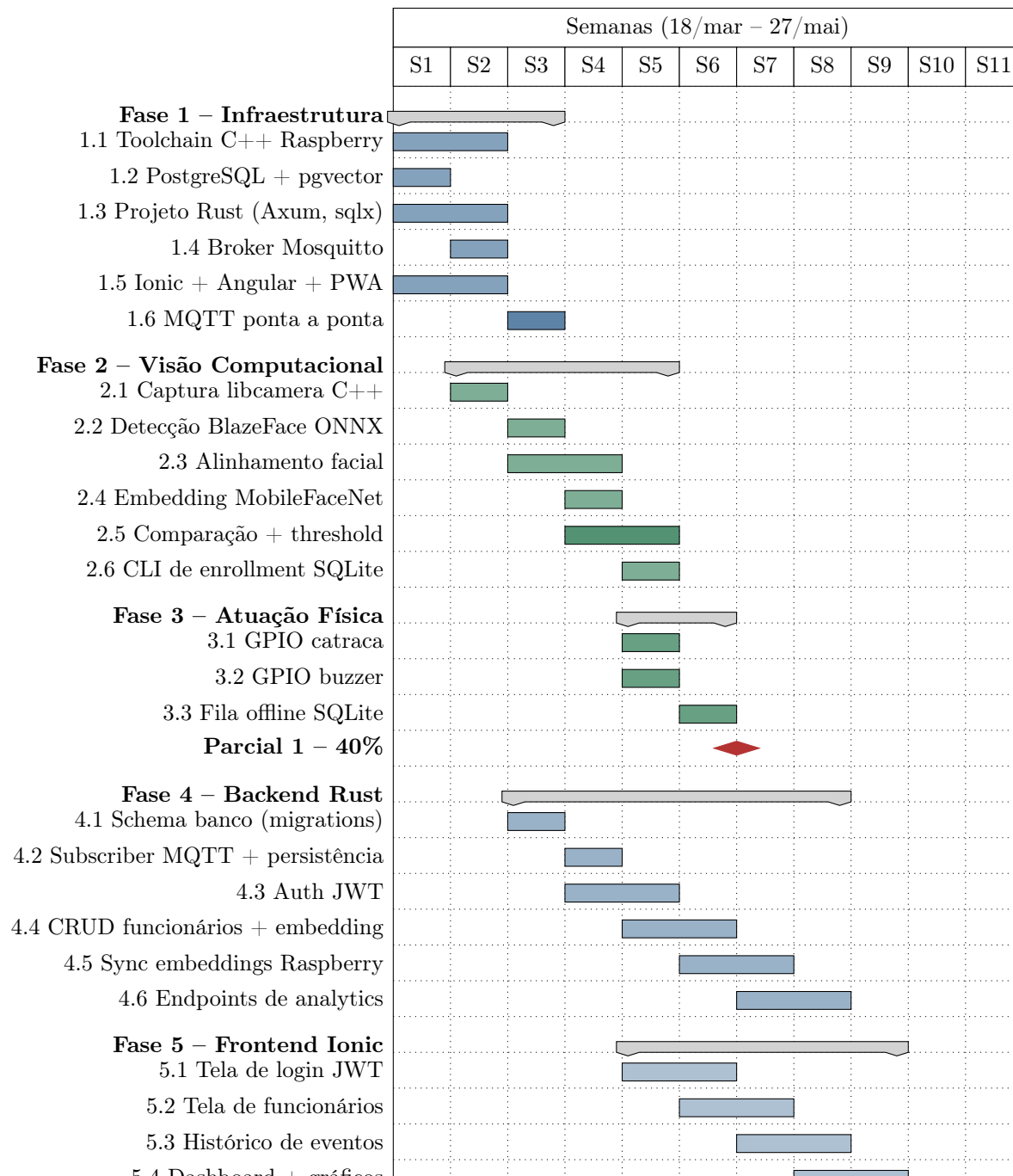
4.2 Cronograma – Diagrama de Gantt

O semestre compreende 11 semanas de desenvolvimento entre a entrega do escopo (18/03) e a entrega final (27/05). O projeto possui três marcos intermediários definidos pelo professor, apresentados na Tabela 4.2.

Tabela 4.2: Marcos do projeto

Data	Marco	Meta de conclusão
18/03	Entrega do escopo	Documento de arquitetura
22/04	Entrega parcial 1	$\approx 40\%$ – Fases 1, 2 e 3 concluídas
13/05	Entrega parcial 2	$> 80\%$ – Fases 4 e 5 concluídas
27/05	Entrega final	Sistema integrado e documentado
03/06	P2	Avaliação individual

O diagrama de Gantt a seguir apresenta a distribuição das atividades ao longo das semanas, com identificação das dependências entre tarefas e dos marcos de entrega.



4.3 Estimativa de Custos

A Tabela 4.3 apresenta os componentes de hardware necessários para o MVP, com preços estimados para o mercado brasileiro em 2026.

Tabela 4.3: Estimativa de custo do hardware para o MVP

Componente	Quantidade	Preço estimado (R\$)
Raspberry Pi 4 Model B – 4 GB	1	650,00
Câmera CSI 5MP (compatível RPi)	1	120,00
Cartão microSD 32 GB (Class 10)	1	40,00
Fonte de alimentação USB-C 5V/3A	1	50,00
Módulo relé 5V (acionamento catraca)	1	15,00
Buzzer ativo 5V	1	8,00
Catraca de barreira (demonstração)	1	180,00
Jumpers e protoboard	1 kit	25,00
Total estimado		R\$ 1.088,00

Os custos de software são nulos: todas as tecnologias utilizadas – Rust, C++, PostgreSQL, Mosquitto, Ionic, Angular e ONNXRuntime – são de código aberto e gratuitas. O servidor local (notebook) já pertence ao grupo, eliminando custo de infraestrutura. Em um cenário de produção, o backend seria hospedado em nuvem, com custo variável conforme o provedor escolhido.

Capítulo 5

Metodologia

5.1 Coleta de Dados e Cadastro de Funcionários

O cadastro de um funcionário no sistema ocorre por meio de uma ferramenta de linha de comando (*enrollment CLI*) desenvolvida em C++. O operador captura múltiplas fotos do funcionário em diferentes condições de iluminação e ângulo. Para cada imagem, o pipeline extrai o embedding via MobileFaceNet. O vetor resultante é armazenado na tabela `embeddings` do PostgreSQL, associado ao identificador do funcionário.

Nenhuma imagem bruta é persistida no banco de dados – apenas os vetores de 128 dimensões. Isso atende diretamente ao princípio de minimização de dados da LGPD.

5.2 Definição do Limiar de Autenticação

O limiar de decisão (distância de cosseno) será ajustado empiricamente. Serão realizados testes comparando:

- Embeddings do mesmo indivíduo em sessões distintas (testes positivos);
- Embeddings de indivíduos diferentes (testes negativos).

O objetivo é encontrar o valor que minimize simultaneamente a Taxa de Falsos Positivos (FAR – *False Accept Rate*) e a Taxa de Falsos Negativos (FRR – *False Reject Rate*). O valor inicial de referência é 0,45, com ajuste conforme os resultados dos testes.

5.3 Testes e Avaliação

A avaliação do sistema considerará:

- **Precisão do reconhecimento:** acurácia, FAR e FRR com os rostos dos integrantes do grupo;

- **Latência:** tempo total desde a captura do frame até a decisão de acesso (meta: <500 ms);
- **Resiliência offline:** eventos gerados sem rede são entregues ao broker ao reconectar;
- **Consistência das métricas:** valores de analytics verificados manualmente contra os registros brutos no banco.

Capítulo 6

Desenvolvimento

6.1 Implementação do Sistema

6.2 Integração dos Módulos

6.3 Desafios Técnicos

Capítulo 7

Resultados

7.1 Desempenho do Modelo

7.2 Tempo de Processamento

7.3 Análise Logística Gerada

Capítulo 8

Discussão

Capítulo 9

Conclusão

Referências

- DENG, J. et al. **ArcFace: Additive Angular Margin Loss for Deep Face Recognition**. CVPR, 2019.
- BAZAREVSKY, V. et al. **BlazeFace: Sub-millisecond Neural Face Detection on Mobile GPUs**. Google Research, 2019.
- CHEN, S. et al. **MobileFaceNets: Efficient CNNs for Accurate Real-Time Face Verification on Mobile Devices**. CCBR, 2018.
- MICROSOFT. **ONNX Runtime: Cross-platform inference accelerator**. Disponível em: <https://onnxruntime.ai>. Acesso em: mar. 2026.
- ECLIPSE FOUNDATION. **Eclipse Mosquitto: An open source MQTT broker**. Disponível em: <https://mosquitto.org>. Acesso em: mar. 2026.
- PGVECTOR. **Open-source vector similarity search for Postgres**. Disponível em: <https://github.com/pgvector/pgvector>. Acesso em: mar. 2026.
- BRASIL. **Lei n. 13.709, de 14 de agosto de 2018**. Lei Geral de Proteção de Dados Pessoais (LGPD). Brasília, DF, 2018.

Apêndice A

Especificações Técnicas

Tabela A.1: Stack tecnológico completo

Camada	Tecnologia	Versão	Função
Edge – linguagem	C++17	GCC 12+	Pipeline de visão e GPIO
Edge – visão	OpenCV	4.8	Captura e pré-processamento
Edge – inferência	ONNXRuntime	1.17	Execução dos modelos ONNX
Edge – álgebra	Eigen3	3.4	Distância de cosseno
Edge – cache	SQLite	3.x	Embeddings e fila offline
Edge – MQTT	libmosquitto	2.x	Publicação de eventos
Broker	Eclipse Mosquitto	2.x	Roteamento MQTT
Backend	Rust + Axum	1.78 / 0.7	API REST
Backend – banco	sqlx	0.7	Queries compile-time
Backend – MQTT	rumqttc	0.24	Subscriber assíncrono
Backend – auth	jsonwebtoken	9.x	JWT HS256
Banco de dados	PostgreSQL + pgvector	16 / 0.6	Persistência + busca vetorial
Frontend	Ionic + Angular	7 / 17	Painel web/mobile PWA
Frontend – gráficos	ngx-echarts	17.x	Visualizações de analytics

Apêndice B

Trechos Relevantes de Código